



Reinforcement Learning

Model-Free Techniques

CS4246/CS5446

Reinforcement Learning and Sequential Decision Making

V 4.1



Topics

- Value Function Approximation (22.4.1 - 22.4.3)
 - Approximating direct utility estimation
 - Approximating temporal difference learning
 - Deep Reinforcement learning
- Policy Search (22.5)
 - REINFORCE
 - Trust Region Policy Optimization (TRPO)
 - Proximal Policy Optimization (PPO)
- Actor-Critic Methods

Recall: Reinforcement Learning (RL)

- Based on:

A Markov decision process (MDP) $M \triangleq (S, A, T, R)$ consisting of:

- A set S of states
- A set A of actions
- T and R are **unknown**

- Prediction (policy evaluation) – Passive Learning:

- Assume policy π
- Solution is an (optimal) utility (value) function: $U: S \rightarrow \mathbb{R}$ or $V: S \rightarrow \mathbb{R}$

- Control (policy optimization) – Active Learning:

- Assume utility function or Q-function (action-utility function)
- Solution is an (optimal) policy: $\pi^*: S \rightarrow A$

Recall: Reinforcement Learning (RL)

- **Passive Learning — fixed policy (policy evaluation)**
 - [MB] Adaptive dynamic programming (ADP)
 - [MF] Monte Carlo prediction (MC): direct value (utility) estimation
 - [MF] Temporal-difference (TD) prediction
- **Active Learning — optimal policy (control)**
 - [MB] Active ADP (model-based control)
 - [MF] Monte Carlo control
 - [MF] Active TD methods (TD control): Q-learning, SARSA

Scaling: The Problem

- Number of states grow exponentially with no. of state variables (features)
- Tabular representation (for utility function and Q -function) scales to tens of thousands of states
 - Backgammon: $\sim 10^{20}$
 - Chess: $\sim 10^{40}$
 - Go: $\sim 10^{172}$
 - Still considered relatively small as compared to real-world problems!
 - Cannot visit all the states infinitely often
- Approaches to scaling up:
 - **Function approximation** – approximating utility (value) functions
 - **Policy search** – systematic search for good policies
 - **Mixed** – combined value approximation and policy search methods



Function Approximation

Linear:

- Approximate Monte Carlo Learning
- Approximate temporal difference (TD) learning

Non-linear:

- Deep neural networks

Function Approximation: The Idea

Suppose that we want to build an agent that could estimate the **value** of a house **based on its size**.

- Approach 1:

- Collect data, put in a table (e.g., Excel sheet)
- If asked, then look up the answer
- This is what we were doing before the Recess week!

- Better approach?

- Collect data, learn a function that fits the data
- Benefits: compact representation, generalization

Function Approximation

- Main Ideas:

- Represents value or Q-function compactly using few parameters.
- Generalizes from limited training states to the full state space.
- In general:

$$\hat{U}_{\theta}(s) = g(s; \theta), \quad \text{where } g \text{ is any differentiable function}$$

- Linear case: as **weighted features**.

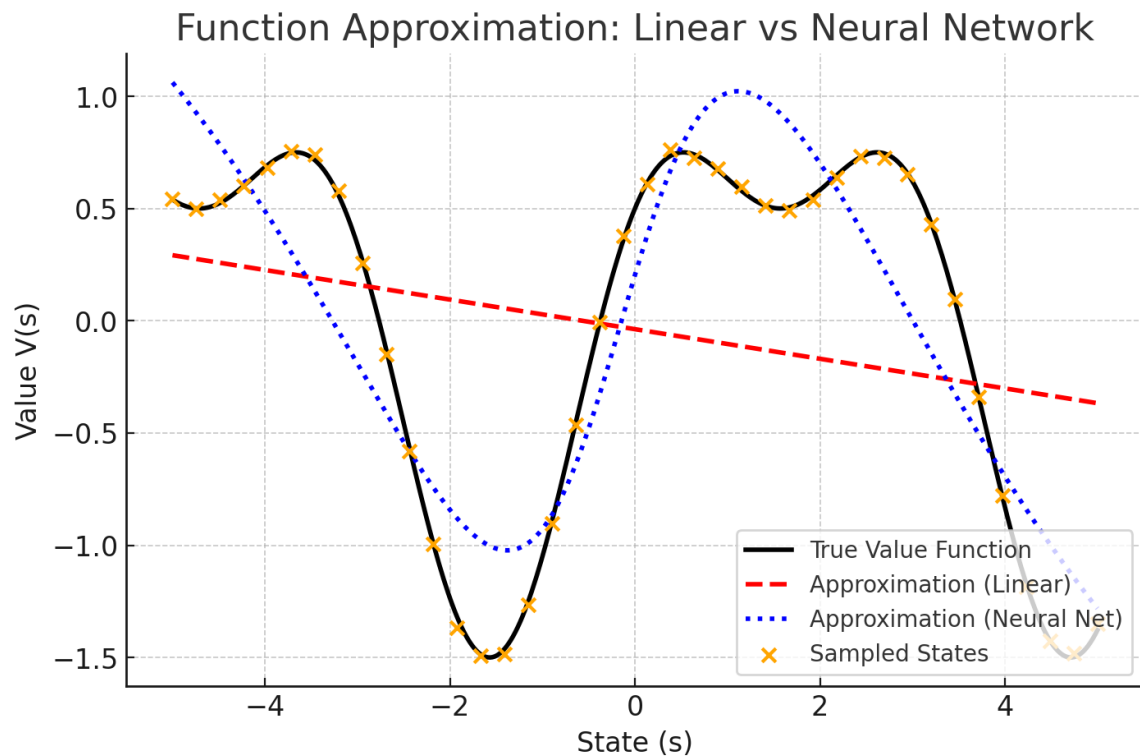
$$\hat{U}_{\theta}(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \cdots + \theta_n f_n(s) \quad \text{What are features?}$$

- Agent learns θ to best approximate the true utility.

- Caveat:

- Too few parameters (n) can cause poor approximation.

Function Approximation





Linear Function Approximation

Approximate Monte Carlo Learning

Approximate temporal difference (TD) learning

Linear Function Approximation

- The idea:

- Use a linear function of features to approximate the utility function.

- General form:

$$\hat{U}_\theta(s) = \theta_0 f_0(s) + \theta_1 f_1(s) + \theta_2 f_2(s) + \cdots + \theta_n f_n(s)$$

where $f_0(s) = 1$: bias feature; $f_i(s)$: features derived from state s ; θ_i : learned weights

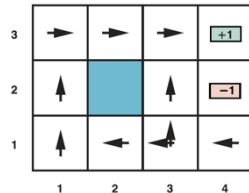
- Let:

- $s = [s_1, s_2, \dots, s_N]$, where each s_i is a component of the state.

- Then:

- $\hat{U}_\theta(s) = \theta_0 + \theta_1 s_1 + \theta_2 s_2 + \cdots + \theta_N s_N$
- Equivalent to the general form: $f_0 = 1, f_i = s_i$

Example: Navigation in Grid World



Source: RN Figure 17.2

- Use:

$$\hat{U}_{\theta}(x, y) = \theta_0 + \theta_1 x + \theta_2 y$$

Compact representation +
generalization

- If $(\theta_0, \theta_1, \theta_2) = (0.5, 0.2, 0.1)$, then:

$$\hat{U}(1,1) = 0.5 + 0.2(1) + 0.1(1) = 0.8$$

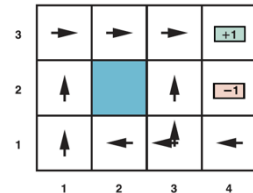
- How to learn θ ?

- Collect sample data from trials: $(x, y, \text{target value})$
- Target values estimate the true utility of each state, based on experience
 - E.g., Monte Carlo returns, TD targets
- Fit θ using linear regression to minimize squared error between predicted and target values.

Approximating Monte Carlo Learning

- Goal:
 - Learn parameters θ to approximate the utility function from experience.
- Learning from Samples:
 - Collect training data from trials: where u_j is the return of state (x_j, y_j)
 $((x_1, y_1), u_1), ((x_2, y_2), u_2), \dots, ((x_n, y_n), u_n)$
 - A supervised learning problem: Fit $\hat{U}_\theta(s)$ to match target values u_j
- Loss function: Per sample
 - $\mathcal{E}_j(s) = \frac{1}{2} \left(\hat{U}_\theta(s) - u_j(s) \right)^2$
- Update rule: Gradient descent
 - $\theta_i \leftarrow \theta_i + \alpha \left(u_j(s) - \hat{U}_\theta(s) \right) \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$
 - This is the Widrow–Hoff Rule (Delta Rule) for online learning.

Approx MC Learning: Grid World



Source: RN Figure 17.2

- Linear Approximation:

- $\hat{U}_\theta(x, y) = \theta_0 + \theta_1 x + \theta_2 y$
- If $\theta_0 = 0.5, \theta_1 = 0.2, \theta_2 = 0.1$, then $\hat{U}(1,1) = 0.5 + 0.2(1) + 0.1(1) = 0.8$

- Parameter updates:

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial \mathcal{E}_j(s)}{\partial \theta_i} = \theta_i + \alpha \left(u_j(s) - \hat{U}_\theta(s) \right) \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

$$\theta_0 \leftarrow \theta_0 + \alpha \left(u_j(s) - \hat{U}_\theta(s) \right)$$

$$\theta_1 \leftarrow \theta_1 + \alpha \left(u_j(s) - \hat{U}_\theta(s) \right) x$$

$$\theta_2 \leftarrow \theta_2 + \alpha \left(u_j(s) - \hat{U}_\theta(s) \right) y$$

Approximating Temporal Difference Learning

- Apply online learning to reduce the temporal difference (TD) error.

- For utility (value) function (TD(0)):

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \hat{U}_\theta(s') - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

Update parameter to
reduce temporal
difference

- For SARSA (on-policy)

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a)] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

- For Q-learning (off-policy): TD target

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \max_a \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a)] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

- Notes:

- Semi-gradient TD: target depends on θ ; treat it as fixed when computing the gradient.

Model-Free Reinforcement Learning

- On-policy TD(0) with linear approximation guaranteed to converge under standard conditions.

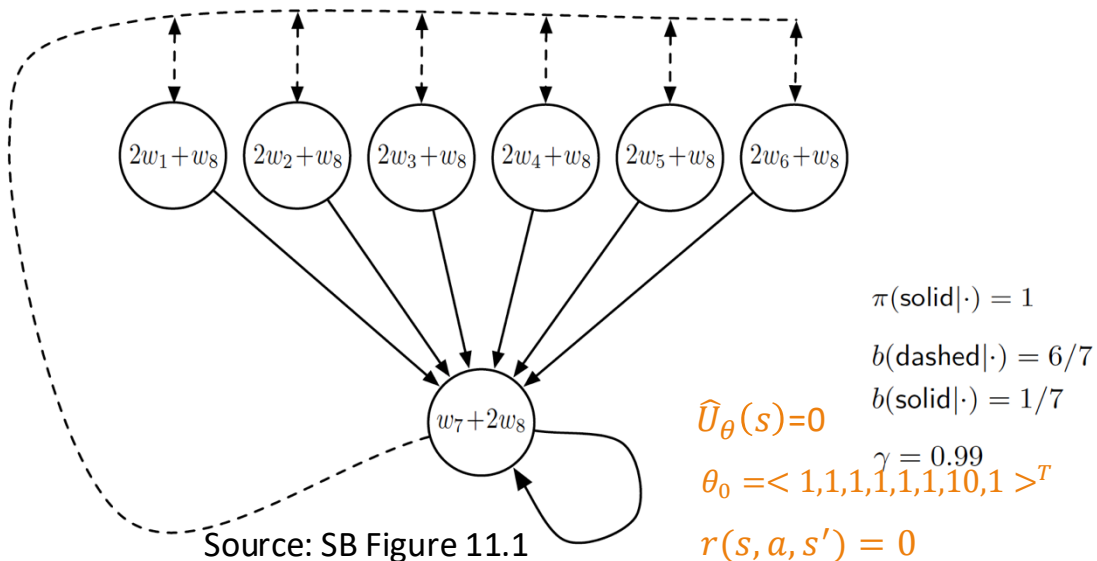
Instabilities in Approximate Utility Learning

- The Deadly Triad

- Function approximation
 - Required in large state spaces (e.g., linear models, deep networks)
 - Updates to one state/action change many others
- Bootstrapping
 - Utility estimates updated using other learned estimates (e.g., TD) instead of full returns.
 - Targets are **moving** because they depend on the same parameters you're learning.
- Off-policy learning
 - Learning from transitions not generated by the policy being evaluated
 - The data distribution does not match the one your target policy will induce.
- Individually each item is fine, but **when all three appear together** learning can become **unstable or even diverge**.
- Classic example: **Baird's counterexample** (linear function approx + off-policy TD) diverges.

Example: Baird's Counterexample

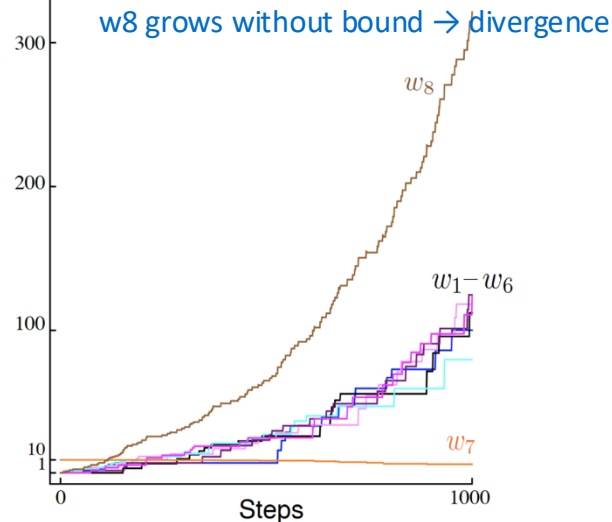
- Off-policy TD with linear function approximation diverges
- Even with a simple setting, semi-gradient TD can fail



Source: SB Figure 11.1

Model-Free Reinforcement Learning

Semi-gradient Off-policy TD



Source: SB Figure 11.2



Catastrophic Forgetting

- What happens:
 - Agent forgets earlier parts of state space
 - Utility estimates degrade in rarely visited regions
- When it happens:
 - Over-training with function approximation
 - Lack of coverage during exploration
- Solution: Experience Replay
 - Store and reuse past transitions
 - Helps maintain accurate utility across all regions



Non-Linear Functional Approximation

Deep reinforcement learning

Deep Reinforcement Learning

- Deep neural network in function approximation

- Uses a deep neural network to approximate utility:

$$\hat{U}_{\theta}(s) = g(s; \theta),$$

where g is a nonlinear function defined by the network

- Learns useful features automatically from raw input
- Final layer often *linear*:

$$\hat{U}_{\theta}(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \dots + \theta_n f_n(s)$$

where $f_i(s)$ are outputs from the last hidden layer

- Parameters θ : all weights in the network
- Gradients computed via backpropagation, as in supervised learning

Deep Reinforcement Learning

- Learning the parameters:

- For utilities:

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \hat{U}_\theta(s') - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

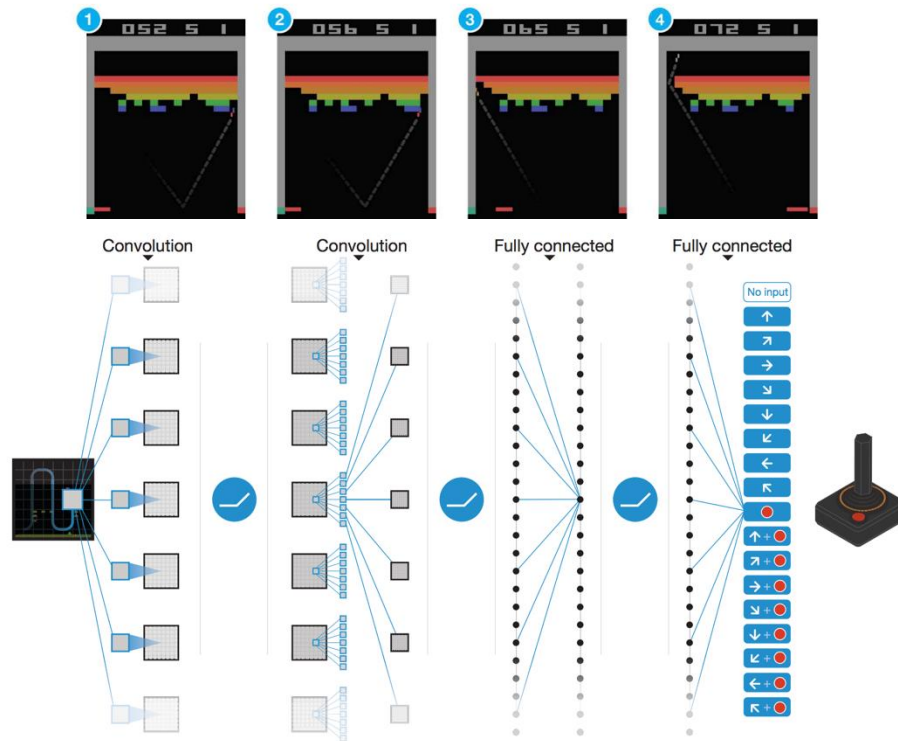
- For Q -learning:

$$\theta_i \leftarrow \theta_i + \alpha \left[R(s, a, s') + \gamma \max_{a'} \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a) \right] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

- Note:

- SARSA is less common in deep RL because its on-policy nature makes it harder to use experience replay effectively.

Deep Q -learning for Atari Games¹



¹Mnih, V., et al., *Human-level control through deep reinforcement learning*. Nature, 2015. **518**(7540): p. 529-533.

Deep Q -Learning (DQN) for Atari Games

- Overview:

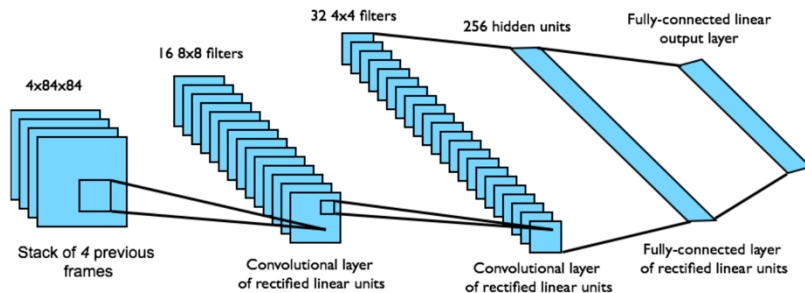
- Learns action-utility values $Q(s, a)$ end-to-end from raw pixels
- Achieved human-level performance on many Atari games (Mnih et al., Nature, 2015)

- Key Features:

- Input: stack of last 4 game frames
- Output: $Q(s, a)$ for 18 joystick actions
- Reward = change in game score

- Architecture (see diagram):

- Convolutional layers extract features from pixel input
- Fully connected layers output Q -values
- Same architecture and hyperparameters used across all games



Network architecture (fixed across all games)

Deep Q-Network (DQN) Algorithm

- Why DQN?

- Standard Q-learning is unstable with deep nets
- DQN introduces two key tricks to stabilize learning:
 1. **Experience Replay**: Breaks correlation in data
 2. **Fixed Q-Targets**: Reduces moving target problem

- Training Loop (see diagram):

1. Take action using ϵ -greedy policy
2. Store (s, a, r, s') in replay buffer
3. Sample minibatch from buffer
4. Compute target: $y = r + \gamma \max_{a'} Q(s', a', \theta^-)$
5. Minimize square loss: $\mathcal{L}(\theta) = (y - Q(s, a; \theta_i))^2$
6. Periodically update target network: $\theta^- \leftarrow \theta$

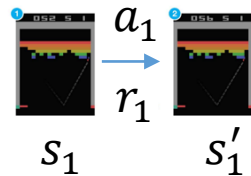
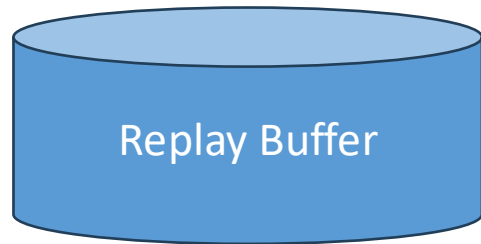
Deep Q-Network (DQN)



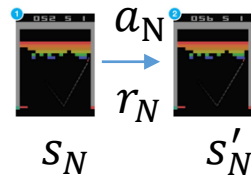
$$\pi_{\epsilon\text{-greedy}}(s, \theta)$$

- With probability ϵ : choose a random action uniformly from all actions
- With probability $1 - \epsilon$: choose a greedy action $a = \operatorname{argmax}_a Q(s, a)$

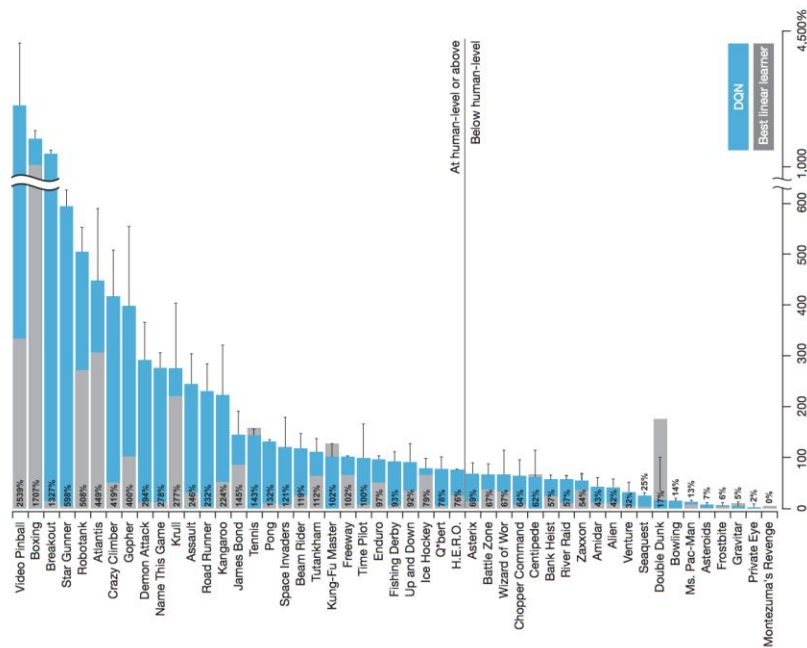
$$\theta_i \leftarrow \theta_i + \alpha \left[R(s, a, s') + \gamma \max_a \hat{Q}_{\theta^{\text{red}}}(s', a') - \hat{Q}_{\theta^{\text{blue}}}(s, a) \right] \frac{\partial \hat{Q}_{\theta}(s, a)}{\partial \theta_i}$$



...



Deep Q -learning for Atari Games¹



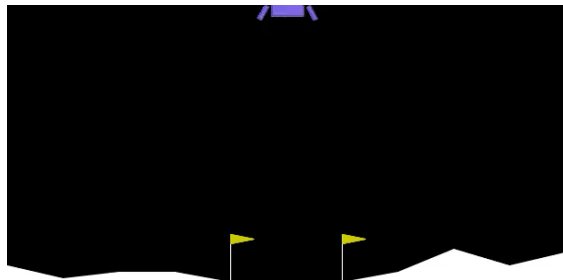
¹Mnih, V., et al., *Human-level control through deep reinforcement learning*. Nature, 2015. **518**(7540): p. 529-533.

Lunar Lander

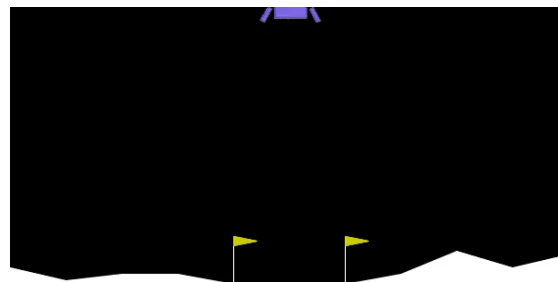
Action Space	<code>Discrete(4)</code>
Observation Space	<code>Box([-2.5 -2.5 -10. -10. -6.2831855 -10. -0. -0.], [2.5 2.5 10. 10. 6.2831855 10. 1. 1.], (8,), float32)</code>
import	<code>gymnasium.make("LunarLander-v3")</code>

Trained DQN with MLP as $Q(s,a)$

Episode #1



Episode #1500



Notes: Variants of Deep Q -Learning

- DQN (reference):

- Target: $r(s, a, s') + \gamma \max_{a'} Q(s', a'; \theta_i^-)$
- Standard DQN uses the target network θ^- to stabilize updates, but may overestimate values.

- Double DQN

- Target: $r(s, a, s') + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta_i); \theta_i^-)$
- Reduces overestimation by separating action selection and evaluation across networks.

- Multi-step learning

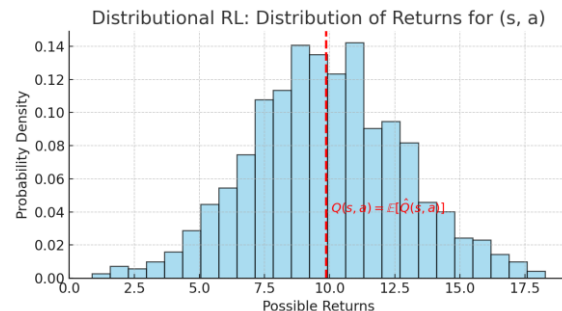
- Target: $r_1 + \gamma r_2 + \dots + \gamma^N \max_{a_N} Q(s_N, a_N; \theta_i^-)$
- Uses multiple future rewards instead of a single step to improve learning signal.

- Distributional RL

- Predicts a distribution of returns, not just the mean. $Q(s, a) = E[\hat{Q}(s, a)]$

- Prioritized replay

- Samples transitions with higher TD error more often, improving data efficiency.



Where DQN struggles?

- DQN needs $\max_a Q(s, a)$. In continuous spaces that argmax is intractable $\Rightarrow$ vanilla DQN doesn't apply.
- DQN must score every action each step; cost and instability grow with $|A|$
- DQN's greedy policy is deterministic. Many tasks benefit from inherent randomness (multi-modal strategies, exploration)



Policy Search

Policy gradient, TPPO, PPO, actor critic

Policy Search

- Definitions:

- Parameterized stochastic policy: $\pi_{\theta}(a|s)$ = probability of choosing action a in state s
- Policy value: $J(\theta) = E_{\tau \sim \pi_{\theta}} [\sum_{t=0}^{T-1} \gamma^t r_t]$ – Expected discounted return when π_{θ} is executed ^[1]
- Policy gradient: $\nabla_{\theta} J(\theta)$ – gradient of policy value w.r.t. parameter vector θ (vector of partial derivatives $[\partial J / \partial \theta_i]$ with the same dimension as θ).

- Goal:

- Maximize policy value $J(\theta)$ via gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Policy gradient

- Iteratively update θ to increase $J(\theta)$.

- [1] Note: Other RL objectives exist, e.g., long-run average reward and risk-sensitive objectives. Here we continue using the cumulative discounted-return objective from the previous lecture.

Policy Gradient

- Policy Value:

$$J(\theta) = \sum_s p_0(s) \sum_a \pi_\theta(a|s) Q^{\pi_\theta}(s, a)$$

where $p_0(s)$ is the initial-state distribution.

- Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[Q^{\pi_\theta}(s, a) \nabla_\theta \log \pi_\theta(a|s)]$$

- Gives direction to adjust θ to improve $J(\theta)$: adjusts the policy toward actions with larger $Q^{\pi_\theta}(s, a)$.
 - $E_{\pi_\theta}[\cdot]$: expectation over state-action visits induced by executing π_θ , with the appropriate discounted visitation weighting.
- Key ideas:
 - $J(\theta)$: how good the policy is.
 - $\nabla_\theta J(\theta)$: how to improve it.
 - Foundation of REINFORCE and Actor–Critic methods.

Differentiable Policy Parameterization

- Deterministic form:

$$\pi_{\theta}(a|s) = \begin{cases} 1, & a = \arg \max_a \hat{Q}_{\theta}(s, a) \\ 0, & \text{otherwise} \end{cases}$$

- Not differentiable for discrete actions $\rightarrow$ gradients not applicable

- Stochastic (Softmax) Policy:

$$\pi_{\theta}(a|s) = \frac{e^{\beta h_{\theta}(s,a)}}{\sum_{a'} e^{\beta h_{\theta}(s,a')}}$$

- β controls “softness” (higher $\rightarrow$ closer to argmax).
- Differentiable $\rightarrow$ enables gradient-based learning.

REINFORCE

- From Policy Gradient Theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s)].$$

- Use G_t as a Monte Carlo sample of $Q^{\pi_{\theta}}(s, a)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

- REINFORCE update rule (stochastic ascent):

$$\theta \leftarrow \theta + \alpha \sum_t \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Return from time t : $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$
- The episode gradient is a sum of contributions from each time step.
- Derived via the log-derivative trick: $\frac{\nabla_{\theta} \pi_{\theta}}{\pi_{\theta}} = \nabla_{\theta} \log \pi_{\theta}$.

High Variance Problem

- Issue:

- Monte Carlo returns (G_t) from episodes can vary widely $\rightarrow$ noisy gradient updates.

Episode	Return (G_t)	Adjusted ($G_t - 100$)	$G_t \nabla_{\theta} \log \pi_{\theta}(a_t s_t)$	$(G_t - 100) \nabla_{\theta} \log \pi_{\theta}(a_t s_t)$
#1	110	+10	132	12
#2	90	-10	-90	10
#3	105	+5	115.5	5.5
#4	100	0	-90	0

- Solution:

Var = 15,275

Var = 28.4

- Subtracting a baseline $B(s)$ keeps the expected gradient unchanged:
- $\sum_a \nabla_{\theta} \pi_{\theta}(a | s) B(s) = B(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) = 0$.

- Effect: lower variance, same expected direction.

Baseline and Advantage Function

- Choose baseline $B(s)$ as the state-utility estimate:

$$B(s) = \hat{U}_{\phi}(s) \approx \hat{U}_{\pi_{\theta}}(s)$$

- Define **advantage function**:

$$A_{\pi_{\theta}}(s, a) = Q_{\pi_{\theta}}(s, a) - U_{\pi_{\theta}}(s).$$

- Policy gradient with baseline:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [A_{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

- Update rule (variance-reduced REINFORCE):

$$\theta \leftarrow \theta + \alpha \sum_t A_{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Bridge forward:

- Using $\hat{U}_{\pi_{\theta}}(s)$ as a learned baseline $\rightarrow$ **Actor-Critic methods**

- When the baseline is learned (a critic), the policy (actor) uses it to reduce variance.

Variance reduction using a Baseline

- Using a baseline function $B(s)$ can reduce variance
- Natural choice: estimated $\hat{U}_{\pi_\theta}(s)$
- The function $A_{\pi_\theta}(s, a) = \hat{Q}_{\pi_\theta}(s, a) - \hat{U}_{\pi_\theta}(s)$ is called the **advantage function**

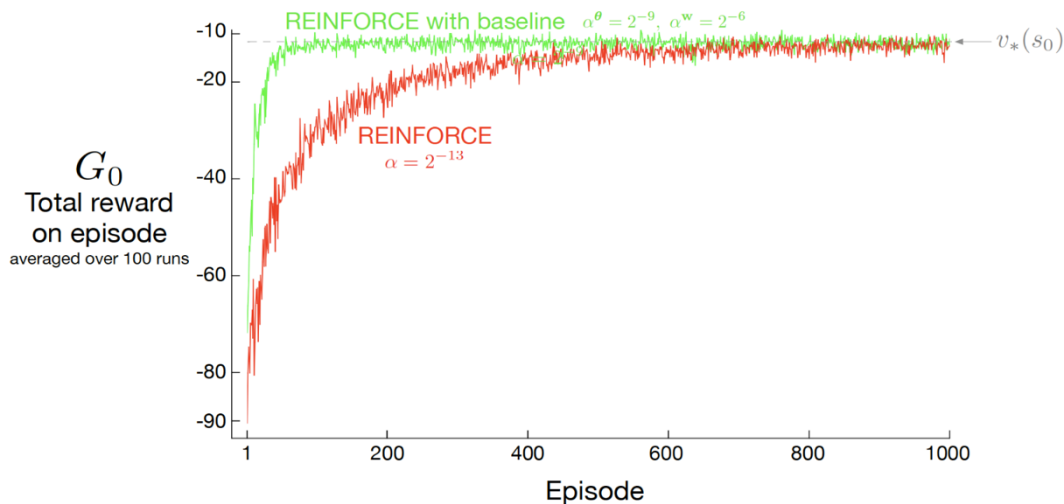


Figure 13.1 Sutton & Barto



Actor-Critic Methods

Combined value-function approximation and policy search

Actor–Critic: The Idea

- Actor–Critic combines policy search and function approximation.
 - **Actor**: parameterized policy $\pi_{\theta}(a | s)$ — selects actions to maximize return.
 - **Critic**: parameterized value function $\hat{U}_w(s)$ or $\hat{Q}_w(s, a)$ — estimates expected return under π_{θ} .
- They are trained together:
 - The critic $\hat{U}_w$ learns to predict returns (function approximation).
 - The actor π_{θ} improves the policy using the critic as a learned baseline (policy search).
- Notes:
 - Actor = policy with parameters θ
 - Critic = value estimator with parameters w
 - Together, they form a function-approximation-based policy search method — a stable, low-variance extension of REINFORCE.

Actor-Critic (TD Version)

- REINFORCE:

- Uses Monte Carlo estimate of advantage $\rightarrow$ high variance

$$A_{\pi_{\theta}}(s, a) = \hat{Q}_{\pi_{\theta}}(s, a) - \hat{U}_{\pi_{\theta}}(s) \approx G_t - \hat{U}_{\phi}(s)$$

- TD Advantage (Actor-Critic):

- Approximate $\hat{Q}_{\pi_{\theta}}(s, a)$ with one-step return:

$$A_{\pi_{\theta}}(s, a) \approx r + \gamma \hat{U}_{\pi_{\theta}}(s') - \hat{U}_{\pi_{\theta}}(s)$$

- Actor Update:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) [r_t + \gamma \hat{U}(s_{t+1}, w) - \hat{U}(s_t, w)]$$

- Critic Update (learn $\hat{U}(s, w)$):

$$\delta_t = r_t + \gamma \hat{U}(s_{t+1}, w) - \hat{U}(s_t, w) \Rightarrow w_{t+1} = w_t + \beta \delta_t \nabla_w \hat{U}(s_t, w)$$

- Notes:

- Actor updates the policy $\pi_{\theta}(a | s)$;
- Critic learns the baseline $\hat{U}(s, w)$ from the TD error $\delta_t \approx A_t$.
- In practice, δ_t serves as the estimated advantage A_t in the actor update

Notes: Variants of Actor-Critic

- A2C – synchronous version, batch actors update together.
- A3C – asynchronous, multiple agents update global parameters.
- ACER – adds experience replay, retrace, and trust-region ideas.
- Many others build on the same actor–critic core (e.g., DDPG, SAC)
- Soft Actor–Critic (SAC) — a modern off-policy algorithm that combines stochastic policy gradients with entropy regularization for stable, sample-efficient, and exploration-driven learning in continuous control tasks.



Advanced Policy Search

Trust Region Policy Optimization (TPRO)

Proximal Policy Optimization (PPO)

Improving Policy Gradients

- Problem:

- Vanilla policy gradients can make unreliable, destructive updates.
- Unstable return $\rightarrow$ bad gradient $\rightarrow$ worse performance
- Need monotonic improvement: every update should improve or maintain performance

- Goal:

- Constrain each update so that

$$J(\theta_{\text{new}}) \geq J(\theta_{\text{old}})$$

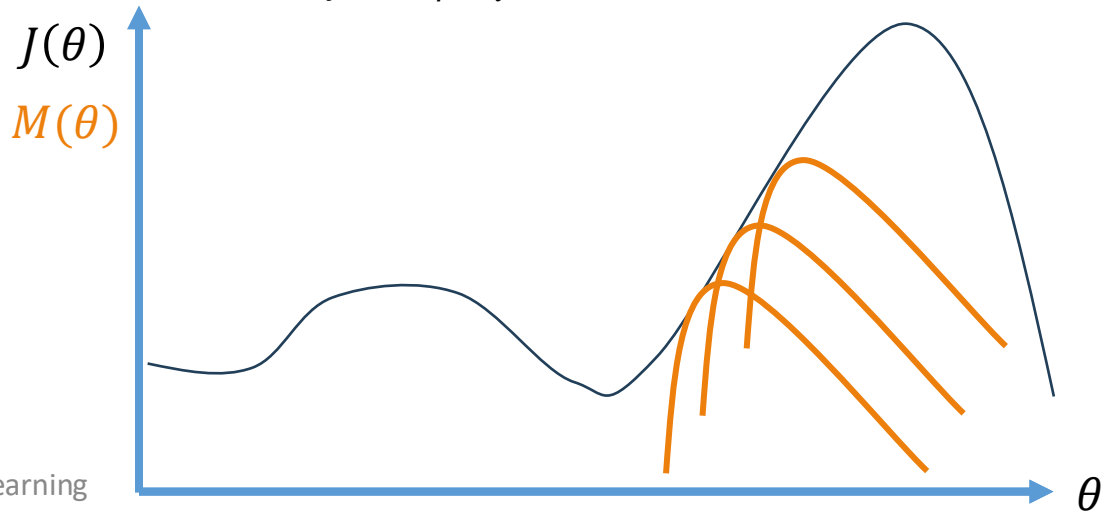
- Intuition:

- Take only safe, trustable steps in policy space — not leaps.

Policy Gradient: Avoiding Bad Updates

- Unstable return $\rightarrow$ bad update $\rightarrow$ unstable return $\rightarrow \dots \rightarrow$ fail
- The Idea: restrict the update of π_θ to only “safe” regions

At i -th iteration, maximize a simpler surrogate objective $M_i(\theta_i)$ that lower-bounds the true objective $\rho(\theta_i)$



Trust Region Policy Optimization (TRPO)

- Idea:

- Keep new policies close to the old one using a trust region constraint.

$$\max_{\theta} \mathcal{L}_{\pi_{\theta}} \quad \text{s.t.} \quad \max_s KL [\pi_{\text{old}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s)] \leq \delta$$

- $KL[\cdot]$ = Kullback–Leibler divergence measures “distance” between policies.
 - Ensures updates do not drift too far, preserving expected improvement.

- Intuition:

- TRPO acts like a speed limit — large steps (big KL-divergence) are unsafe and can ruin performance.

Proximal Policy Optimization

- Simplifies TRPO with a clipped surrogate loss:

$$r_t(\pi) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)},$$

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\pi)A_t, \text{clip}(r_t(\pi), 1 - \epsilon, 1 + \epsilon)A_t)].$$

- When $r_t(\pi)$ moves outside $[1 - \epsilon, 1 + \epsilon]$, the objective stops increasing
→ prevents excessively large updates.
- **Intuition:**
 - $r_t(\pi)$ measures how much we changed the policy.
 - $\mathcal{L}^{CLIP}$ ensures we only change it within safe, stable limits.
 - PPO is TRPO made practical — clipping keeps updates gentle and stable without solving a constrained optimization each time.

Proximal Policy Optimization (PPO)

- PPO: an actor–critic algorithm with a clipped policy-gradient objective.
 - The critic estimates advantages; the actor updates its policy via a safe, bounded step.

- Actor:

- A parameterized policy $\pi_{\theta}(a | s)$ — the actor — decides which action to take.
It's trained via a policy-gradient objective (the clipped surrogate loss):

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\pi)A_t, \text{clip}(r_t(\pi), 1 - \epsilon, 1 + \epsilon)A_t)],$$

- where A_t is the advantage estimate.

- Critic:

- A value network $\hat{U}_w(s)$ (or $V_w(s)$) — the critic — estimates expected returns to compute A_t :

$$A_t = r_t + \gamma \hat{U}_w(s_{t+1}) - \hat{U}_w(s_t)$$

- or with multi-step returns for smoother estimation.

- Note:

- The critic provides a baseline that reduces variance, and the actor uses this feedback to adjust the policy — exactly the actor–critic loop.

How PPO extends standard actor–critic

- Main idea:

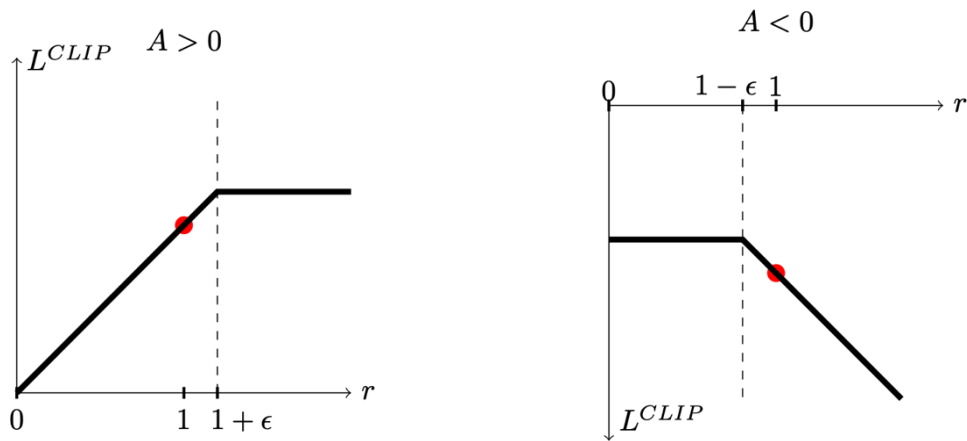
- PPO adds a trust-region–like constraint to stabilize the actor’s updates:

it prevents large destructive policy shifts by clipping the ratio

$$r_t(\pi) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}.$$

- This makes PPO a stable variant of actor–critic, just as TRPO is a theoretical trust-region version.

Proximal Policy Optimization



- With the clipped surrogate objective, policy stops increasing the objective when:

$$r_t(\pi) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$$

moves too far outside the range $[1 - \epsilon, 1 + \epsilon]$

Modern Model-Free Reinforcement Learning

Concept	Core Idea	Key Algorithm
Policy Gradient	Directly optimize the policy $\pi_{\theta}(a s)$ from sampled returns	REINFORCE
Variance Reduction	Learn a value baseline $\hat{U}(s, w)$ to stabilize updates	Actor–Critic
Stable Improvement	Constrain updates for monotonic policy performance	TRPO
Practical Stability	Simplify trust-region updates with clipping	PPO
Exploration & Robustness	Encourage entropy / better signals	SAC, Reward Shaping

Homework

- Readings:

- RN: 22.4.1, 22.4.2, 22.4.3, 22.5

- References:

- SB: Chapter 13

- [SB] Sutton, R. S. and A. G. Barto. Reinforcement Learning: An introduction. 2nd ed. MIT Press, 2018, 2020

[Book website: <http://incompleteideas.net/book/the-book.html>]

[e-Book for personal use: <http://incompleteideas.net/book/RLbook2020.pdf>]

- Online resources on reinforcement learning:

- Silver, D. Lectures on Reinforcement Learning. 2015;
Available from: <https://www.davidsilver.uk/teaching/>.

References

(Journal articles publicly available online or through NUS Library e-Resources)

- Deep reinforcement learning (DeepMind series):
 - DQN: Mnih, V., et al., Human-level control through deep reinforcement learning. *Nature*, 2015. 518(7540): p. 529-533.
 - AlphaGo: Silver, D., et al., Mastering the game of Go with deep neural networks and tree search. *Nature*, 2016. 529: p. 484+.
 - AlphaGo Zero: Silver, D., et al., Mastering the game of Go without human knowledge. *Nature*, 2017. 550: p. 354+.
 - MuZero: Schrittwieser, J., et al., *Mastering Atari, Go, chess and shogi by planning with a learned model*. *Nature*, 2020. **588**(7839): p. 604-609.
- Policy optimization (search)
 - Schulman, J., et al., Trust Region Policy Optimization, in *Proceedings of the 32nd International Conference on Machine Learning*, B. Francis and B. David, Editors. 2015, PMLR: Proceedings of Machine Learning Research. p. 1889--1897.
 - Schulman, J., et al., Proximal Policy Optimization Algorithms. *CoRR*, 2017. abs/1707.06347. Accessible from: <http://arxiv.org/abs/1707.06347>